Exam: Time Series Visualization with Bokeh

This exam tests your ability to visualize time series data using the Bokeh library. You will be working with the "Daily Minimum Temperatures in Melbourne" dataset. For each question, provide the Python code using Bokeh to generate the requested visualization.

Dataset: "daily-minimum-temperatures-in-melbourne.csv"

```
import pandas as pd
from bokeh.plotting import figure, show
from bokeh.io import output_notebook
from bokeh.models import (
    ColumnDataSource,
    HoverTool,
    DatetimeTickFormatter,
    NumeralTickFormatter,
)
from bokeh.layouts import row, column
from bokeh.transform import factor_cmap

output_notebook()  # Enable Bokeh output in Jupyter Notebook

# Load the Dataset
df = pd.read_csv("daily-minimum-temperatures-in-melbourne.csv")

# Rename columns for clarity
df.columns = ['Date', 'Temperature']

# Convert the 'Date' column to datetime format
df['Date'] = pd.to_datetime(df['Date'])

# Remove '?' from the 'Temperature' column and convert to numeric
df['Temperature'] = df['Temperature'].astype(str).str.replace('?', '', regex=False)
df['Temperature'] = pd.to_numeric(df['Temperature'])
```

In [80]:
```
from bokeh.plotting import figure, show
from bokeh.io import output_notebook
from bokeh.models import (
    ColumnDataSource,
    HoverTool,
    DatetimeTickFormatter,
    NumeralTickFormatter,
)
from bokeh.layouts import row, column
from bokeh.transform import factor_cmap
import pandas as pd

output_notebook()  # Enable Bokeh output in Jupyter Notebook

# Load the Dataset
df = pd.read_csv("../datasets/daily-minimum-temperatures-in-melbourne.csv")

# Now you can proceed with your Bokeh plotting code!
print(df.head()) # Just to see if the dataframe loaded correctly


# Rename columns for clarity
df.columns = ['Date', 'Temperature']

# Convert the 'Date' column to datetime format
df['Date'] = pd.to_datetime(df['Date'])

# Remove '?' from the 'Temperature' column and convert to numeric
df['Temperature'] = df['Temperature'].astype(str).str.replace('?', '', regex=False)
df['Temperature'] = pd.to_numeric(df['Temperature'])

df
```

BokehJS 3.8.2 successfully loaded.

```
         Date DailyTemperature
0  1981-01-01             20.7
1  1981-01-02             17.9
2  1981-01-03             18.8
3  1981-01-04             14.6
4  1981-01-05             15.8
```

Out[80]:

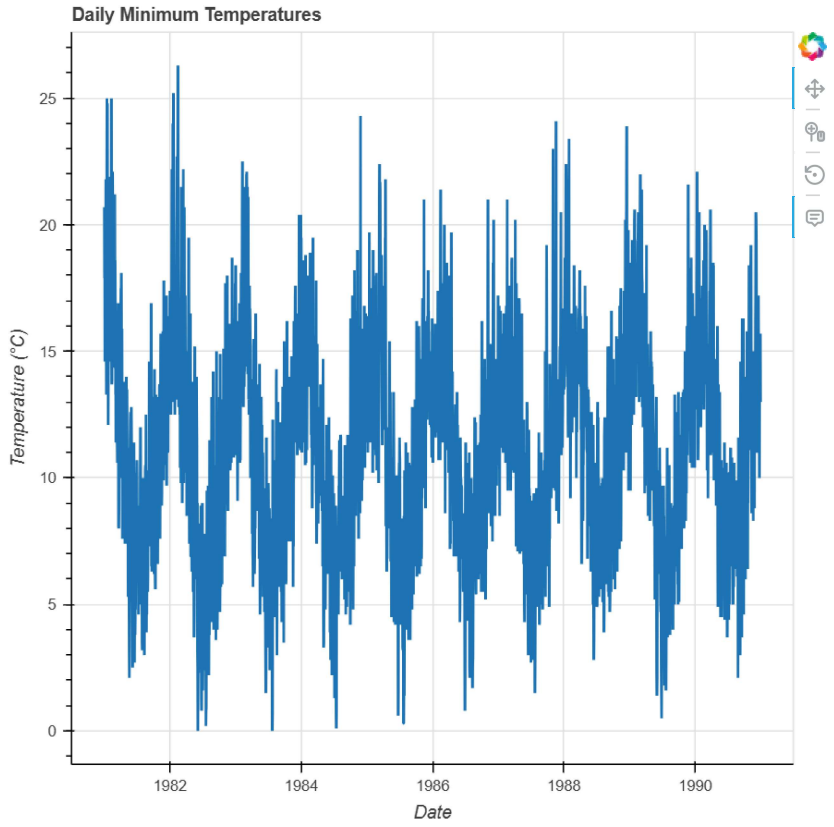
	Date	Temperature
0	1981-01-01	20.7
1	1981-01-02	17.9
2	1981-01-03	18.8
3	1981-01-04	14.6
4	1981-01-05	15.8
...	...	...
3645	1990-12-27	14.0
3646	1990-12-28	13.6
3647	1990-12-29	13.5
3648	1990-12-30	15.7
3649	1990-12-31	13.0

3650 rows × 2 columns

Question 1: Basic Time Series Line Plot

1. Create a basic line plot showing the daily minimum temperature over time.
   - Use the 'Date' column on the x-axis and the 'Temperature' column on the y-axis.
   - Set the plot title to "Daily Minimum Temperatures".
   - Label the x-axis as "Date" and the y-axis as "Temperature (°C)".
   - Add tooltips to display the date and temperature when hovering over the line.
   - Enable pan, wheel zoom, and reset tools.

In [70]:
```
source_q1 = ColumnDataSource(df)

p1 = figure(
    title="Daily Minimum Temperatures",
    x_axis_type="datetime",
    tools="pan,wheel_zoom,reset"
)

r1 = p1.line(
    x="Date",
    y="Temperature",
    source=source_q1,
    line_width=2
)

hover1 = HoverTool(
    renderers=[r1],
    tooltips=[("Date", "@Date{%F}"), ("Temperature", "@Temperature{0.0} °C")],
    formatters={"@Date": "datetime"},
    mode="vline"
)

p1.add_tools(hover1)
p1.xaxis.axis_label = "Date"
p1.yaxis.axis_label = "Temperature (°C)"

show(p1)
```

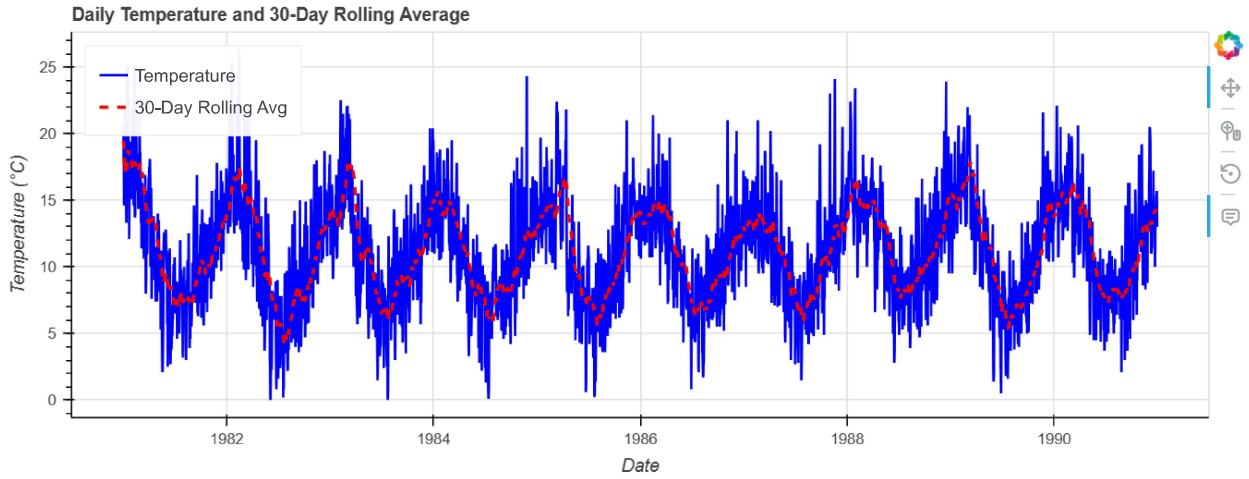


Question 2: Rolling Average 2. Calculate the 30-day rolling average of the daily minimum temperature and plot it alongside the original temperature data.

* Create a new column 'Rolling_Avg' in the DataFrame containing the 30-day rolling average.
* Plot both the original 'Temperature' and the 'Rolling_Avg' on the same plot.
* Use different colors and line styles to distinguish between the two.
* Add a legend to the plot to label the lines.
* Add tooltips to display the date, original temperature, and rolling average.

In [68]:
```
df["Rolling_Avg"] = df["Temperature"].rolling(window=30, min_periods=1).mean()

source_q2 = ColumnDataSource(df)

p2 = figure(
    title="Daily Temperature and 30-Day Rolling Average",
    x_axis_type="datetime",
    tools="pan,wheel_zoom,reset",
    width=900,
    height=350
)

r2a = p2.line(
    x="Date",
    y="Temperature",
    source=source_q2,
    line_width=1.8,
    color="blue",
    legend_label="Temperature"
)

r2b = p2.line(
    x="Date",
    y="Rolling_Avg",
    source=source_q2,
    line_width=2.4,
    color="red",
    line_dash="dashed",
    legend_label="30-Day Rolling Avg"
)

hover2 = HoverTool(
    renderers=[r2a, r2b],
    tooltips=[
        ("Date", "@Date{%F}"),
        ("Temperature", "@Temperature{0.0} °C"),
        ("Rolling Avg", "@Rolling_Avg{0.0} °C")
    ],
    formatters={"@Date": "datetime"},
    mode="vline"
```

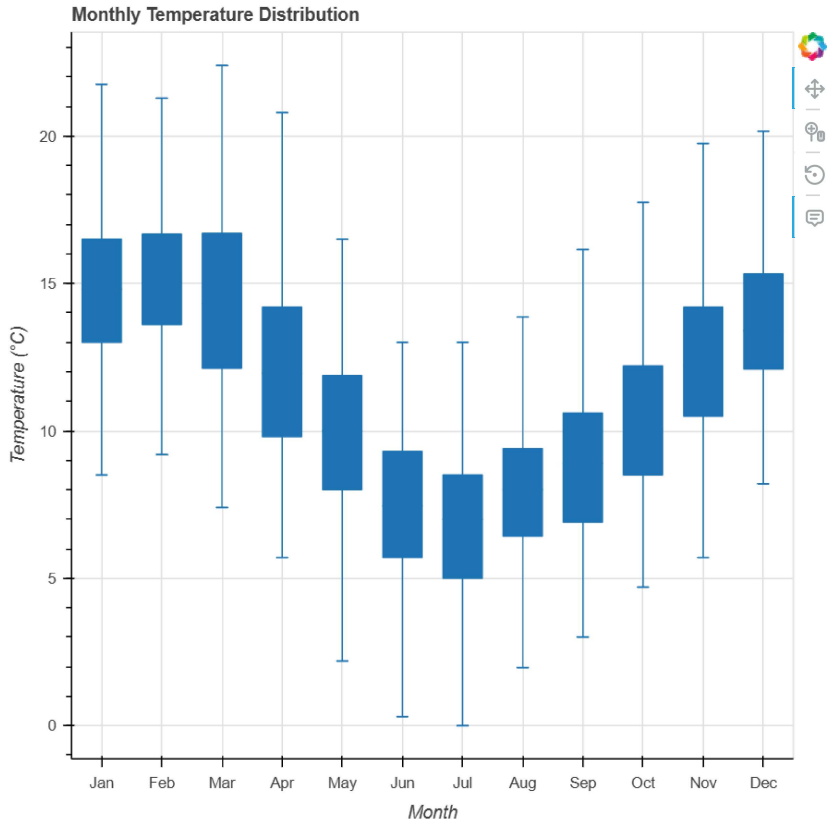
```
)

p2.add_tools(hover2)
p2.legend.location = "top_left"
p2.xaxis.axis_label = "Date"
p2.yaxis.axis_label = "Temperature (°C)"

show(p2)
```



Question 3: Monthly Box Plots 3. Create box plots to visualize the distribution of temperatures for each month.

* Extract the month from the 'Date' column and create a new 'Month' column.
* Group the data by 'Month' and prepare it for plotting.
* Use Bokeh's box plot elements to visualize the distribution.
* Label the x-axis with month names and the y-axis with "Temperature (°C)".
* Add tooltips to display the month and relevant statistical values (min, max, media

In [81]:
```
df["Month"] = df["Date"].dt.month_name().str.slice(0, 3)
month_order = ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"]

g = df.groupby("Month")["Temperature"]
monthly_stats = g.describe().reset_index()
monthly_stats["q1"] = g.quantile(0.25).values
monthly_stats["q2"] = g.quantile(0.50).values
monthly_stats["q3"] = g.quantile(0.75).values
monthly_stats["upper"] = (monthly_stats["q3"] + 1.5 * (monthly_stats["q3"] - monthly_stats["q1"])).clip(upper=monthly_stats|
monthly_stats["lower"] = (monthly_stats["q1"] - 1.5 * (monthly_stats["q3"] - monthly_stats["q1"])).clip(lower=monthly_stats|
monthly_stats["Month"] = pd.Categorical(monthly_stats["Month"], categories=month_order, ordered=True)
monthly_stats = monthly_stats.sort_values("Month")
monthly_stats["Month"] = monthly_stats["Month"].astype(str)

source_q3 = ColumnDataSource(monthly_stats)

p3 = figure(
    title="Monthly Temperature Distribution",
    x_range=month_order,
    tools="pan,wheel_zoom,reset"
)

r3a = p3.segment(x0="Month", y0="upper", x1="Month", y1="q3", source=source_q3)
r3b = p3.segment(x0="Month", y0="lower", x1="Month", y1="q1", source=source_q3)
p3.vbar(x="Month", width=0.65, top="q3", bottom="q2", source=source_q3)
p3.vbar(x="Month", width=0.65, top="q2", bottom="q1", source=source_q3)
p3.rect(x="Month", y="lower", width=0.18, height=0.01, source=source_q3)
p3.rect(x="Month", y="upper", width=0.18, height=0.01, source=source_q3)

hover3 = HoverTool(
    tooltips=[
        ("Month", "@Month"),
        ("Min", "@min{0.0} °C"),
        ("Max", "@max{0.0} °C"),
        ("Median", "@q2{0.0} °C")
    ]
)

p3.add_tools(hover3)
p3.xaxis.axis_label = "Month"
p3.yaxis.axis_label = "Temperature (°C)"

show(p3)
```

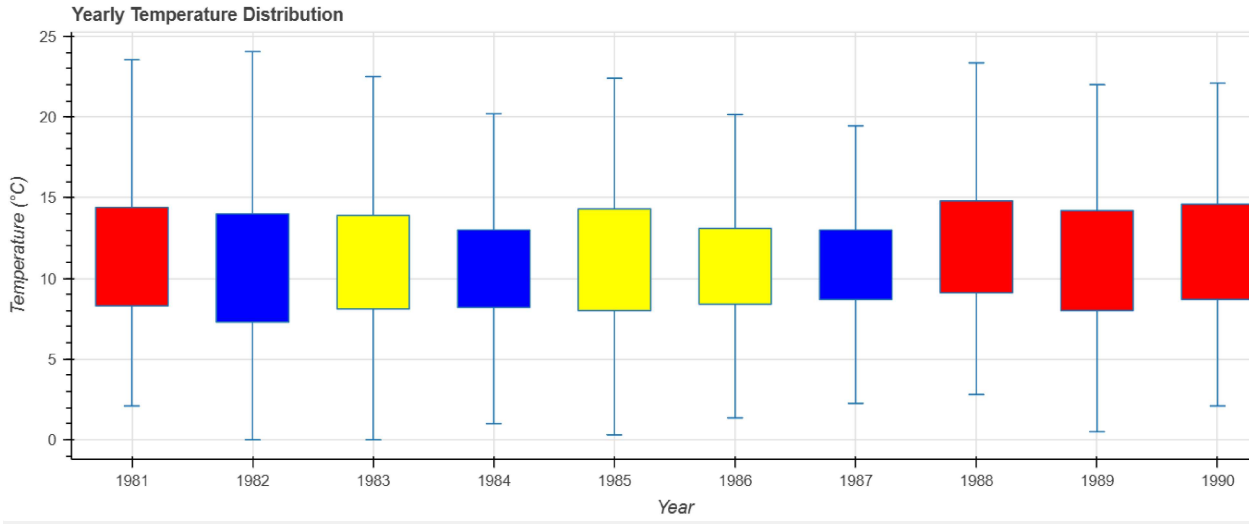


4. Create box plots to visualize the distribution of temperatures for each year, and use color mapping to highlight temperature variations.

   - Extract the year from the 'Date' column and create a new 'Year' column.
   - Group the data by 'Year' and prepare it for plotting.
   - Use Bokeh's box plot elements to visualize the distribution for each year.
   - Label the x-axis with the 'Year' and the y-axis with "Temperature (°C)".
   - Use `factor_cmap` to color the boxes based on the median temperature of each year.
   - Add tooltips to display the year and relevant statistical values (min, max, median, etc.).
   - Enable pan, wheel zoom, and reset tools.

In [82]:
```
df["Year"] = df["Date"].dt.year.astype(str)

g = df.groupby("Year")["Temperature"]
yearly_stats = g.describe().reset_index()
yearly_stats["q1"] = g.quantile(0.25).values
yearly_stats["q2"] = g.quantile(0.50).values
yearly_stats["q3"] = g.quantile(0.75).values
yearly_stats["upper"] = (yearly_stats["q3"] + 1.5 * (yearly_stats["q3"] - yearly_stats["q1"])).clip(upper=yearly_stats["max"
yearly_stats["lower"] = (yearly_stats["q1"] - 1.5 * (yearly_stats["q3"] - yearly_stats["q1"])).clip(lower=yearly_stats["min"

q_low = yearly_stats["q2"].quantile(0.33)
q_high = yearly_stats["q2"].quantile(0.66)
yearly_stats["median_level"] = pd.cut(
    yearly_stats["q2"],
    bins=[-1e9, q_low, q_high, 1e9],
    labels=["Low", "Medium", "High"],
    include_lowest=True
).astype(str)

year_order = yearly_stats["Year"].tolist()
source_q4 = ColumnDataSource(yearly_stats)
color_map_q4 = factor_cmap("median_level", palette=["blue", "yellow", "red"], factors=["Low", "Medium", "High"])

p4 = figure(
    title="Yearly Temperature Distribution",
    x_range=year_order,
    tools="pan,wheel_zoom,reset",
    width=950,
    height=380
)

p4.segment(x0="Year", y0="upper", x1="Year", y1="q3", source=source_q4)
p4.segment(x0="Year", y0="lower", x1="Year", y1="q1", source=source_q4)
r4 = p4.vbar(
    x="Year",
    width=0.6,
    top="q3",
```

```
    bottom="q1",
    source=source_q4,
    fill_color=color_map_q4
)
p4.rect(x="Year", y="lower", width=0.12, height=0.01, source=source_q4)
p4.rect(x="Year", y="upper", width=0.12, height=0.01, source=source_q4)

hover4 = HoverTool(
    renderers=[r4],
    tooltips=[
        ("Year", "@Year"),
        ("Min", "@min{0.0} °C"),
        ("Max", "@max{0.0} °C"),
        ("Median", "@q2{0.0} °C"),
        ("Q1", "@q1{0.0} °C"),
        ("Q3", "@q3{0.0} °C"),
        ("Level", "@median_level")
    ]
)

p4.add_tools(hover4)
p4.xaxis.axis_label = "Year"
p4.yaxis.axis_label = "Temperature (°C)"

show(p4)
```



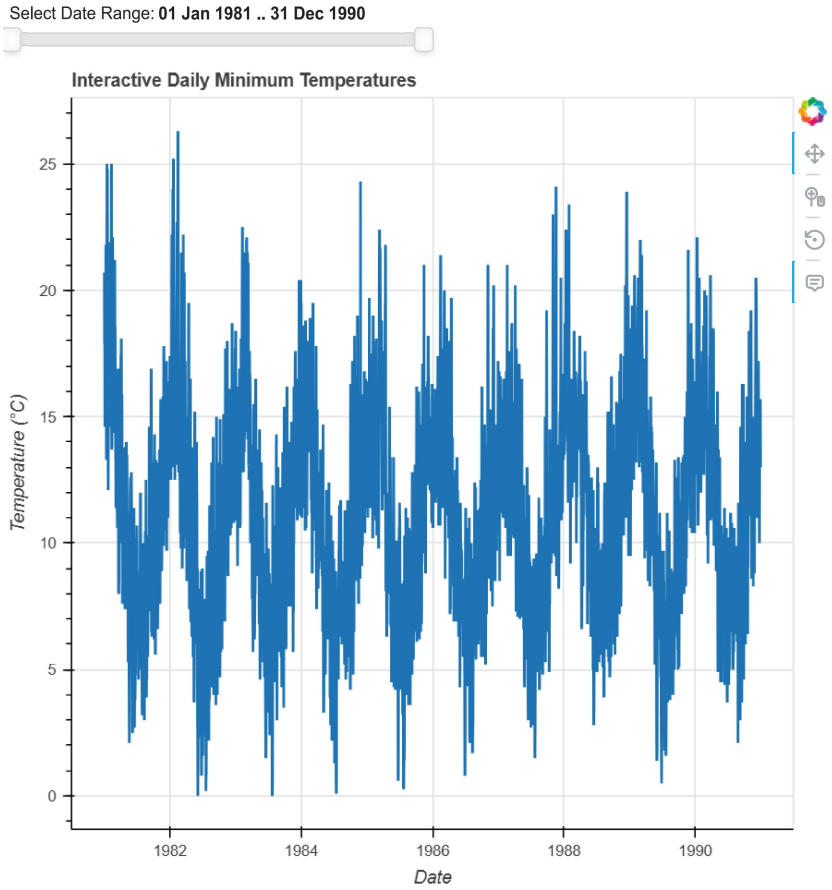
Question 5: Interactive Time Range Selection

5. Create an interactive line plot where the user can select a specific time range to view using a date range slider.

- Create a basic line plot of 'Temperature' over 'Date'.
- Implement a date range slider using Bokeh widgets to allow users to select a start and end date.
- Update the plot dynamically based on the selected date range.
- Add tooltips to display the date and temperature.
- Enable pan, wheel zoom, and reset tools.

In [88]:
```
from bokeh.models import DateRangeSlider

source_q5 = ColumnDataSource(df)

p5 = figure(
    title="Interactive Daily Minimum Temperatures",
    x_axis_type="datetime",
    tools="pan,wheel_zoom,reset"
)

p5.line(
    x="Date",
    y="Temperature",
    source=source_q5,
    line_width=2
)

hover5 = HoverTool(
    tooltips=[("Date", "@Date{%F}"), ("Temperature", "@Temperature{0.0} °C")],
    formatters={"@Date": "datetime"},
    mode="vline"
)

p5.add_tools(hover5)
p5.xaxis.axis_label = "Date"
```

```
p5.yaxis.axis_label = "Temperature (°C)"

date_slider = DateRangeSlider(
    title="Select Date Range",
    start=df["Date"].min(),
    end=df["Date"].max(),
    value=(df["Date"].min(), df["Date"].max())
)

date_slider.js_link("value", p5.x_range, "start", attr_selector=0)
date_slider.js_link("value", p5.x_range, "end", attr_selector=1)

show(column(date_slider, p5))
```



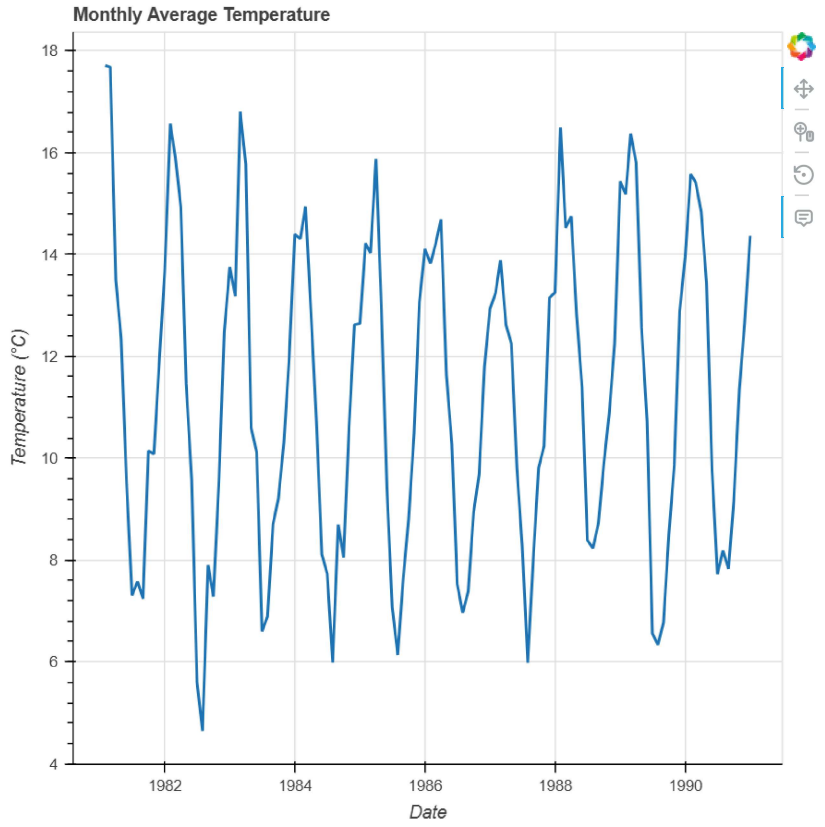
Question 6: Time Series Decomposition Visualization

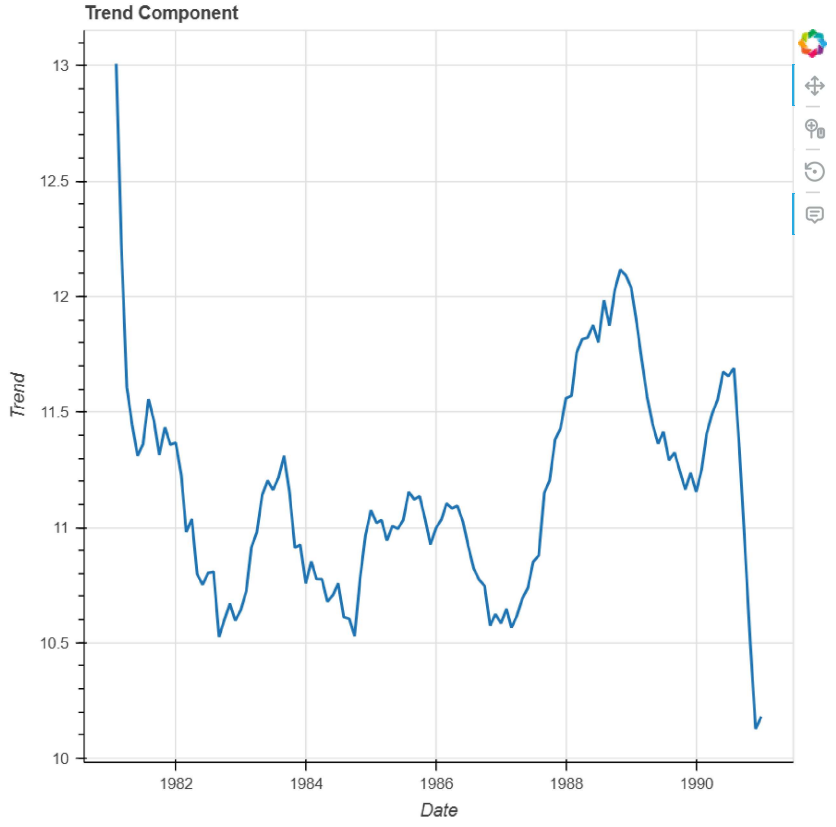
6. Perform a simple time series decomposition to visualize the trend and seasonality components of the temperature data.

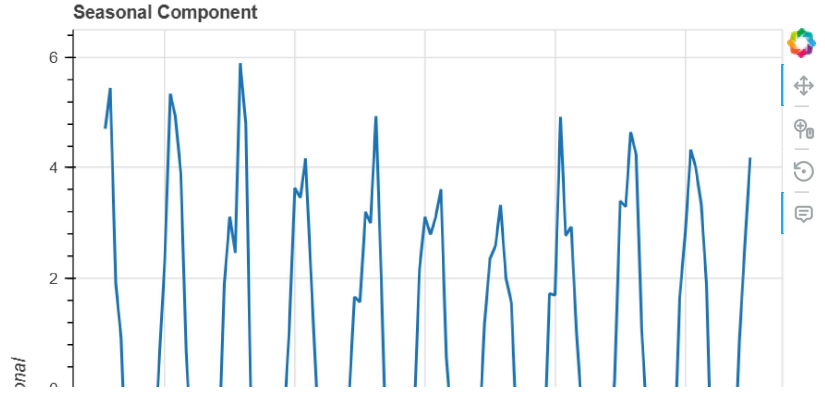
- Resample the data to monthly frequency and calculate the monthly average temperature.
- Use a simple moving average to estimate the trend component.
- Calculate the seasonal component by subtracting the trend from the original monthly data.
- Create three separate Bokeh plots: one for the original monthly data, one for the trend, and one for the seasonal component.
- Ensure the plots are aligned and share the same x-axis (Date).
- Add tooltips to each plot to display the date and corresponding value.
- Enable pan, wheel zoom, and reset tools for each plot.

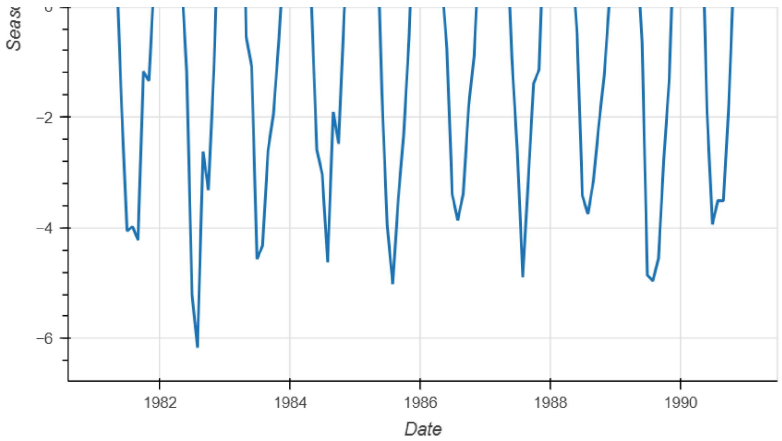
In [84]:
```
monthly = df.set_index("Date")["Temperature"].resample("ME").mean().reset_index()
monthly["Trend"] = monthly["Temperature"].rolling(window=12, min_periods=1, center=True).mean()
monthly["Seasonal"] = monthly["Temperature"] - monthly["Trend"]

source_q6 = ColumnDataSource(monthly)

p6_1 = figure(
    title="Monthly Average Temperature",
    x_axis_type="datetime",
    tools="pan,wheel_zoom,reset"
)
p6_1.line("Date", "Temperature", source=source_q6, line_width=2)
h61 = HoverTool(
    tooltips=[("Date", "@Date{%F}"), ("Monthly Temp", "@Temperature{0.0} °C")],
    formatters={"@Date": "datetime"},
    mode="vline"
)
p6_1.add_tools(h61)
p6_1.xaxis.axis_label = "Date"
p6_1.yaxis.axis_label = "Temperature (°C)"
```

```
p6_2 = figure(
    title="Trend Component",
    x_axis_type="datetime",
    tools="pan,wheel_zoom,reset",
    x_range=p6_1.x_range
)
p6_2.line("Date", "Trend", source=source_q6, line_width=2)
h62 = HoverTool(
    tooltips=[("Date", "@Date{%F}"), ("Trend", "@Trend{0.0} °C")],
    formatters={"@Date": "datetime"},
    mode="vline"
)
p6_2.add_tools(h62)
p6_2.xaxis.axis_label = "Date"
p6_2.yaxis.axis_label = "Trend"

p6_3 = figure(
    title="Seasonal Component",
    x_axis_type="datetime",
    tools="pan,wheel_zoom,reset",
    x_range=p6_1.x_range
)
p6_3.line("Date", "Seasonal", source=source_q6, line_width=2)
h63 = HoverTool(
    tooltips=[("Date", "@Date{%F}"), ("Seasonal", "@Seasonal{0.0} °C")],
    formatters={"@Date": "datetime"},
    mode="vline"
)
p6_3.add_tools(h63)
p6_3.xaxis.axis_label = "Date"
p6_3.yaxis.axis_label = "Seasonal"

show(column(p6_1, p6_2, p6_3))
```


Monthly Average Temperature
Temperature (°C)
18
16
14
12
10
8
6
4
1982
1984
1986
1988
1990
Date


Trend Component
Trend
13
12.5
12
11.5
11
10.5
10
1982
1984
1986
1988
1990
Date


Seasonal Component
6
4
2


−2
−4
−6
1982
1984
1986
1988
1990
Date